

Code Sample

Selected Replication Code

Anonymous

CODING SAMPLE: SHORT COPY

These excerpts are selected from the replication code for the paper. Selected outputs appear at the end of the document.

Survey-weighted enrollment model

File: R/selection/estimate_selection_probit.R

```
selection__probit__variables <- function(selection__data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion__all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion__district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion__all_kids, exclusion__district), names(selection__data))
}

# ' project the selection sample to the model's actual dependency surface
# '
# ' Keeping unrelated Block-5 schooling attributes in the fitted survey design makes
# ' the serialized model change whenever those attributes change, even though they
# ' are absent from the probit formula. A narrow model frame lets {targets} skip the
# ' expensive AME branch when upstream changes do not affect estimation.
project_selection__model_data <- function(selection__data) {
  selection__data <- safe_df(selection__data)
  design <- c("enrolled", "FSU_SL_NO", "weight", "STATE", "STRATUM", "SUB_STRATUM_NO")
  keep <- unique(c(design, selection__probit__variables(selection__data)))
  selection__data[intersect(keep, names(selection__data))]
}

# ' estimate selection probit
# '
estimate_selection__probit <- function(selection__data, cfg) {
```

```

if (!"enrolled" %in% names(selection__data) || all(is.na(selection__data$enrolled))) {
  return(list(status = "out_of_active_pipeline", reason = "No_enrolled_variable."))
}
covars <- selection__probit__variables(selection__data)
if (!length(covars)) {
  return(list(status = "out_of_active_pipeline", reason = "No_probit_covariates."))
}
f__probit <- stats::reformulate(covars, response = "enrolled")
if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
  old_options <- options(survey.lonely.psu = "average")
  on.exit(options(old_options), add = TRUE)

  design <- build_survey_design_selection(selection__data)
  if (!is.null(design)) {
    out <- fit_selection__probit(design, f__probit)
    return(stabilize_selection__model_formula(out, f__probit))
  }
}
out <- stats::glm(f__probit, data = selection__data, family = stats::binomial(link = "probit"))
stabilize_selection__model_formula(out, f__probit)
}

#' store a durable selection-model formula
#'
#' Programmatically fitted models otherwise retain a call to the local symbol
#' `f__probit`. Packages which reconstruct model data from the saved call cannot
#' resolve that symbol after the model is serialized by targets. Embed the
#' formula object in the call and retain the existing audit attribute.
stabilize_selection__model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection_model_formula_must_inherit_from_formula.", call. = FALSE)
  }
  if (!is.null(model$call)) model$call$formula <- formula
  attr(model, "selection__probit_formula") <- formula
  model
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection__df) {
  psu <- first_col(selection__df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection__df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection__df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  selection__df$survey_strata <- interaction(selection__df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),

```

```

    data = selection__df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection__design, f__probit) {
  survey::svyglm(f__probit, design = selection__design, family = stats::quasibinomial(link = "probit"))
}

```

Average marginal effects on the model estimation sample

File: R/selection/compute_average_marginal_effects.R

```

ame__model__weight <- function(model__data, model__weights = NULL) {
  weight__cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier", "(weights)"), names(
    model__data))
  if (length(weight__cols)) {
    return(suppressWarnings(as.numeric(model__data[[weight__cols[[1]]]])))
  }

  if (!is.null(model__weights) && length(model__weights) == nrow(model__data)) {
    return(suppressWarnings(as.numeric(model__weights)))
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has__explicit__weights`.
ame__model__data__and__weights <- function(model) {
  model__data <- as.data.frame(stats::model.frame(model))
  model__weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame__model__weight(model__data, model__weights)

  if (!is.null(weight) && length(weight) == nrow(model__data) && any(is.finite(weight) & weight
    != 1)) {
    model__data$.ame__weight <- weight
    return(list(data = model__data, wts = ".ame__weight", has__explicit__weights = TRUE))
  }

  list(data = model__data, wts = FALSE, has__explicit__weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg__slopes <- function(model, model__data, wts, numderiv = NULL) {

```

```

args <- list(
  model = model,
  newdata = model_data,
  wts = wts,
  vcov = TRUE,
  type = "response"
)
if (!is.null(numderiv)) args$numderiv <- numderiv

withCallingHandlers(
  do.call(marginaleffects::avg__slopes, args),
  warning = function(w) {
    msg <- conditionMessage(w)
    explicit_weight_warning <- grepl(
      "normally_good_practice_to_specify_weights_using_the`wts`_argument",
      msg,
      fixed = TRUE
    )
    if (explicit_weight_warning && !identical(wts, FALSE)) {
      invokeRestart("muffleWarning")
    }
  }
)
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame__model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}

```

Montiel Olea–Pflueger effective-F calculation

File: R/iv/weak_identification.R

```

mop_effective_f <- function(
  model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA__real__, critical_value = NA__real__, p.value = NA__real__,
      effective_df = NA__real__, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }
}

```

```

if (inherits(model, "ivreg")) {
  return(unavailable("Montiel_Olea-Pflueger_effective_F_requires_an_ivreg_model."))
}
if (!requireNamespace("momentfit", quietly = TRUE)) {
  return(unavailable("Package 'momentfit' is not installed."))
}
if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
  return(unavailable("Montiel_Olea-Pflueger_tau_and_size_must_be_valid_probabilities."))
}

terms <- parse_iv_formula_terms(model)
if (is.null(terms)) {
  return(unavailable("Could not parse IV formula for Montiel_Olea-Pflueger_effective_F."))
}
endogenous <- setdiff(terms$regressors, terms$instruments)
if (length(endogenous) != 1L) {
  return(unavailable("Montiel_Olea-Pflueger_effective_F_requires_exactly_one_endogenous_regressor."
  ))
}

formula <- stats::formula(model)
regression_formula <- stats::as.formula(
  call("~", formula[[2L]], formula[[3L]][[2L]]),
  env = environment(formula)
)
instrument_formula <- stats::as.formula(
  call("~", formula[[3L]][[3L]]),
  env = environment(formula)
)
needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
data <- iv_analysis_frame(analysis_data, needed)
missing <- setdiff(needed, names(data))
if (length(missing)) {
  return(unavailable(paste(
    "Missing variables for Montiel_Olea-Pflueger_effective_F:",
    paste(missing, collapse = ", ")
  )))
}
moment_data <- data[needed]
if (any(!stats::complete.cases(moment_data))) {
  return(unavailable("Montiel_Olea-Pflueger_analysis_data_contain_incomplete_model_rows."))
}

stored_cluster <- attr(model, "cluster_state", exact = TRUE)
cluster <- if (
  length(stored_cluster) == stats::nobs(model) &&
  !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
) {
  as.vector(stored_cluster)
} else {

```

```

  iv__model__cluster(model, data)
}
covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance__options <- if (is.null(cluster)) {
  list(type = "HC0")
} else {
  # momentfit's clustered moment covariance is based on sandwich::meatCL()
  # and supports HC0. Keep its package-native finite-cluster correction
  # rather than altering the existing HC1 Wald-F diagnostic to force equality.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HC0", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({
  moment__model <- momentfit::momentModel(
    regression__formula,
    instrument__formula,
    data = moment__data,
    vcov = covariance,
    vcovOptions = covariance__options
  )
  momentfit::MOPtest(
    moment__model,
    tau = tau,
    size = size,
    estMethod = "TSLS",
    simplified = TRUE,
    print = FALSE
  )
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff"])))
names(values) <- c("statistic", "critical__value", "p.value", "effective__df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite effective-F diagnostics."))
}

c(
  as.list(values),
  list(tau = tau, size = size, status = "estimated", reason = NA__character__)
)
}

```

Linked state-sector price series

File: R/prices/build_temporal_price_series.R

```

build_temporal_price_series <- function(
  price_sources,
  switch_date = as.Date("2013-01-01"),
  overlap_start = as.Date("2013-01-01"),
  overlap_end = as.Date("2014-12-01"),
  minimum_link_months = 6L,
  pre_switch_start = NULL,
  pre_switch_end = NULL) {
  switch_date <- price_boundary(switch_date)
  old <- select_pre_2013_price_series(price_sources)
  new <- select_post_2013_price_series(price_sources)
  links <- summarise_price_links(old, new, overlap_start, overlap_end)

  minimum_link_months <- as.integer(minimum_link_months)
  if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_
    months < 1L) {
    stop("minimum_link_months must be one positive integer.", call. = FALSE)
  }

  old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])

  linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"),
    drop = FALSE]
  linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
  missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
  if (any(missing_key)) {
    bad <- old_groups[missing_key, , drop = FALSE]
    stop(
      "Pre-2013 price series lack a sufficient direct link to CPI-R/U:",
      paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
      call. = FALSE
    )
  }

  old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
  old <- old[old$period < switch_date, , drop = FALSE]
  if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
    if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
      stop("Both pre_switch_start and pre_switch_end are required when trimming the old price
        series.", call. = FALSE)
    }
    pre_switch_start <- price_boundary(pre_switch_start)
    pre_switch_end <- price_boundary(pre_switch_end)
    if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
      stop("The pre-switch production window must end before the switch date.", call. = FALSE)
    }
    old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop =
      FALSE]
  }
  old$index_unlinked <- old$index

```

```

old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]
new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
new$index_unlinked <- new$index

columns <- c(
  "state_code", "sector", "period", "index", "index_unlinked", "price_source",
  "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
)
index <- rbind(old[columns], new[columns])
index$year <- as.integer(format(index$period, "%Y"))
index$month <- as.integer(format(index$period, "%m"))
index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
rownames(index) <- NULL
validate_temporal_price_chain(index, switch_date)

structure(
  list(index = index, links = links, switch_date = switch_date),
  class = "emi_temporal_price_series"
)
}

```

Selected Outputs

The tables and figures below are generated by the analyses from which the preceding excerpts are drawn.

Selected enrollment-model average marginal effects

Selected average slopes and discrete comparisons from the survey-weighted enrollment model.

Covariate or comparison	Estimate	SE
Religion: Muslim (ref: Hindu)	-0.125	0.013
Social group: Scheduled Tribe (ref: Other)	-0.101	0.019
Social group: Scheduled Caste	-0.074	0.015
Social group: Other Backward Class	-0.029	0.012
Distance 1–2km (ref: <1km)	-0.076	0.044

Weak-instrument-robust long-run inference

Long-run IV estimates, effective first-stage F statistics, and Anderson–Rubin acceptance sets.

Outcome	2SLS estimate	SE	Effective F	AR 95% set
2022-23 long change	0.117	0.152	0.686	[grid<= -0.193146, -0.0714641] U [0.025109, 0.193146 <=grid]
2023-24 long change	0.137	0.234	0.414	[grid<= -0.185982, -0.0511449] U [0.0148785, 0.185982 <=grid]

Consumption-series consistency

Validation of the harmonized consumption series used in the later-outcome analysis.

